Архитектура и запуск

Две подсистемы в одном процессе

Engine.portal.create_app создаёт FastAPI, применяет LocalAccess, регистрирует страницы сводного аудита и подключает NiceGUI через ui.run_with. Engine.portal.launch запускает Uvicorn на loopback. Windows начинает с start.cmd и Engine/start-portal.ps1, Linux с start.sh.

Границы хранения

Engine/config.py выбирает search.sqlite3 из HACKATON_DB_PATH. Engine/crawl_store.py: initialize_db создаёт схему и выполняет совместимые миграции, Engine/importer.py: seed_if_empty импортирует учреждения из задания. ai_audit_web/paths.py определяет HACKATON_QUICK_DIR, db.init создаёт audit.db. Реестры сопоставляются по URL через integration и roster, числовые ID двух баз независимы.

Архив и повторяемость

Снимки page включают дату и HTML. page_render хранит данные браузерного захвата. Индексы, вложения и оценки ссылаются на конкретный снимок. Часть исходных файлов хранится в соседней .audit-assets. Для резервной копии используйте SQLite backup и копию каталогов материалов при остановленных сборщиках.

Запуск в разработке

Установите requirements.txt в виртуальное окружение, выполните python -m Engine.site_server --port 8765. Для изолированного теста задайте отдельные HACKATON_DB_PATH и HACKATON_QUICK_DIR. python -m unittest discover -s Engine запускает тесты. HACKATON_TEST_BROWSER=1 включает браузерный сценарий, HACKATON_TEST_LOCAL_QWEN=1 требует живую модель на loopback.

Сбор и доказательства

Маршрут страницы

Engine/loadPage.py: crawl_organization отвечает за начальный обход организации. navigation.py обнаруживает ссылки и категории, site_navigator.py ищет навигатор, news_window.py ограничивает период новостей. browser_capture.py: BrowserCapture сохраняет HTML после исполнения сценариев, DOM, экранные снимки и PDF. crawl_store.record_fetch записывает попытку HTTP, успешный page и обнаруженные ссылки.

Дополнительные документы

backfill_materials.seed и process_org ведут очередь material_backfill. attached_files.download_pending получает вложения, проверяет повторное использование и учитывает временную блокировку. document_links.py, document_inventory.py, schedule_evidence.py описывают каталог и доступные материалы расписания.

Правила и сотрудники

home_content.py, home_events.py, home_styles.py и garden_home.py разбирают главную. home_terms.py управляет словарём. staff_extract.py и staff_store.py извлекают и записывают карточки, staff_render.py оценивает видимость, staff_duplicates.py и staff_cross_org.py связывают похожие наблюдения. Совпадение имени между учреждениями остаётся поводом для сверки.

Даты и источники

news_cards.py и news_order.py разбирают превью и порядок. event_quarters.py и timeline_audit.py оценивают кварталы и даты, factcheck_sources.py ограничивает список независимых источников, factcheck_batch.py ведёт очередь сверки.

Автоматизация

audit_worker.run_once выполняет ограниченную партию, записывает audit_worker_cycle и освобождает межпроцессную блокировку. worker_schedule.py управляет заданием Windows. retention.py и retention_records.py применяют срок хранения заменённых данных. Ручные выборки проходят через portal_jobs.preview/start и operation_log.

Модель, отчёты и интерфейс

Модель сводного аудита

model_settings.py хранит профили и флаг разрешения, ключ передаётся в системное хранилище keyring. model_gateway.py формирует совместимый запрос и проверяет ответ. deferred_checks.py задаёт вопросы, page_evidence.py готовит материал, review_contract.py задаёт контракт. model_batch.batch_assess планирует и запускает вопросы. Уже сохранённый результат привязан к снимку, вопросу, профилю и отпечатку входа.

Экспресс-проверка

ai_audit_web/core.py: audit_site выбирает сеть или кэш и выполняет правила; vision.py отвечает за фото. operations.start создаёт задание, сохраняет этапы в своей базе и записывает результат учреждения. reports.latest и select выбирают последние результаты и фильтруют замечания. llm.summarize и explain отправляют замечания на разбор через общий профиль.

Сводный интерфейс

web/home.html отвечает за выбор инструмента и статус, web/index.html и web/app.js — за обзор, учреждения и результаты. URL хранит фильтры, hash выбирает вкладку или карточку. dashboard_data.dashboard и institution формируют JSON из SQLite, dashboard_pdf.build_pdf получает ту же выборку. include_quality управляет только дополнительной таблицей PDF.

Администрирование и NiceGUI

web/admin.html и admin.js получают состояние и токен, отправляют команды, опрашивают прогресс. site_server.update_settings проверяет действие и вызывает модуль сервера. ai_audit_web/app.py регистрирует /quick, /quick/run, /sites, /site/{id}, /report, /quick/ai, /rules, /settings. События кнопок NiceGUI выполняются на сервере.

Переносная версия

Engine.jury_export.write_snapshot переносит разрешённые поля уже вычисленных результатов в web/jury.html. JSON вшит в HTML, текст выводится через DOM textContent. Страница может открываться с диска без API и Python. Локальные токены, файлы SQLite и архивные документы для этого не нужны.

Технологии и назначение

Python и SQLite

Python 3.11+ исполняет сервер, обработчики, задания и CLI. sqlite3 хранит историю, связи и транзакции. OS file locks и SQL-блокировки защищают записи между процессами. asyncio и потоки разделяют обслуживание интерфейса и долгие операции.

FastAPI, Uvicorn, NiceGUI

FastAPI обслуживает JSON и PDF. Uvicorn исполняет ASGI. NiceGUI реализует быстрый инструмент на серверных компонентах и WebSocket. Страницы сводного аудита используют HTML, CSS, JavaScript и Fetch API.

Получение материалов

urllib и HTTP-клиенты получают сайты и API, Beautiful Soup и lxml разбирают HTML/XML. Playwright запускает Chromium для вычисленного CSS и печати. tldextract используется в проверках доменов. defusedxml обрабатывает XML ограниченным парсером.

Файлы и изображения

pypdfium2 читает и растеризует PDF. openpyxl и xlrd читают электронные таблицы, legacy-doc помогает с устаревшим форматом Word. Pillow, NumPy и OpenCV используются для декодирования, размеров, сопоставлений и анализа изображений. ReportLab собирает PDF и встраивает кириллические шрифты.

Локальная и внешняя генерация

LM Studio и Ollama предоставляют локальный API. Совместимый внешний сервис настраивается через base URL и свой ключ. Для /chat/completions доступны JSON по схеме, JSON-объект и текстовая инструкция. Результат зависит от модели и доступности материала. Списки пакетов приведены в requirements.txt и requirements-render.txt.

HTTP API • чтение

GET /healthz и /api/access

Ответ содержит ok, service, admin, ai_status. Эти маршруты подтверждают доступность портала и установленный статус профиля. Для реального запроса к модели используйте POST verify.

GET /api/dashboard

Возвращает as_of, filters, options, grading, summary, institutions, findings. Например /api/dashboard?kind=school&status=issue&detector=ai. Повторяющиеся параметры и списки через запятую поддерживаются для многозначных фильтров. q, date_from и date_to принимают одно значение; формат даты YYYY-MM-DD.

GET /api/institution/{id}

Возвращает подробности по архивному ID, категории и историю. Несуществующее активное учреждение получает HTTP 404. quick_id при наличии указывает на карточку экспресс-проверки.

GET /report.pdf

Возвращает application/pdf и Content-Disposition: attachment. Пример /report.pdf?organization_id=1,2&status=issue&include_quality=1. По умолчанию дополнительная таблица отключена. Ограничение параллельной генерации может вернуть HTTP 503.

GET /api/admin/state, /api/admin/progress, /api/operations

state возвращает реестр, профили без ключей, настройки и токен команд. progress включает job, cycle и manual_cycle. operations отдаёт сокращённое состояние каждого режима для главной страницы. Отображаемые состояния ручного задания: idle, running, completed, error, interrupted.

HTTP API • команды

Правила запроса

POST /api/admin/{action} принимает объект JSON, заголовки Content-Type: application/json, Origin с адресом портала и X-Audit-Token из state. Ошибки параметров дают 400 с полем error, проверка происхождения — 403. В обычном режиме LocalAccess разрешает локальные Host и Origin для HTTP и WebSocket.

plan → analyze → resume

plan: {"ids":[1,2],"mode":"download_algorithm"}. Ответ содержит ticket, pages, empty, gaps и показатели ИИ. analyze получает те же ids, mode и ticket, возвращает id задания. Допустимы algorithm, ai, download, download_algorithm, download_ai. resume получает {"id":1}. Клиент опрашивает progress до завершения.

profile, disable-ai, verify, benchmark

profile получает profile (provider, endpoint, model, timeout, response_mode, disable_thinking), enabled и необязательный secret. disable-ai и verify получают {}. benchmark с decline:true фиксирует отказ, иначе делает замер. Валидность формата проверяется по реальному ответу.

settings, cycle, schedule, retention

settings принимает объект settings с числовыми лимитами. cycle с {} запускает одну партию без ИИ. schedule принимает operation: status, enable или disable. retention принимает preview:true для оценки либо enabled и days для сохранения политики.

roster, words, sources, factcheck

roster поддерживает add с record, import с format и content, remove с id, clear с confirmed:true. words принимает operation (add/remove), category и word. sources получает operation и url. factcheck принимает enabled. POST /api/run и /api/run_cache запускают быстрые сетевые или кэшированные проверки и возвращают статус запуска.

Показ и восстановление

Изоляция

Engine/jury_portal.py запускает копию портала только на 127.0.0.1:8766. KONTUR_DEMO_DIR и HACKATON_DB_PATH должны указывать на изолированный каталог и его search.sqlite3, HACKATON_QUICK_DIR — на data/quick внутри копии. Исходный локальный портал сохраняет проверки Host и Origin.

Вход и транспорт

JuryEntry проверяет Host, Origin, приглашение и сессионную cookie. POST /demo-login получает invite. Приватная ссылка из архива передаёт код во фрагменте URL. Сквозь SSH публикуется только демонстрационный веб-портал; модель отвечает локальному серверу. При завершении процесса сессии просмотра истекают.

Возврат настроек

demo_settings.baseline сохраняет настройки копии перед первым показом. POST /api/admin/restore-demo восстанавливает профиль ИИ, настройки цикла, словарь, разрешённые источники, хранение и настройки правил экспресс-аудита. Быстрые и сводные отчёты сохраняются. Операция требует токен и свободные блокировки заданий.

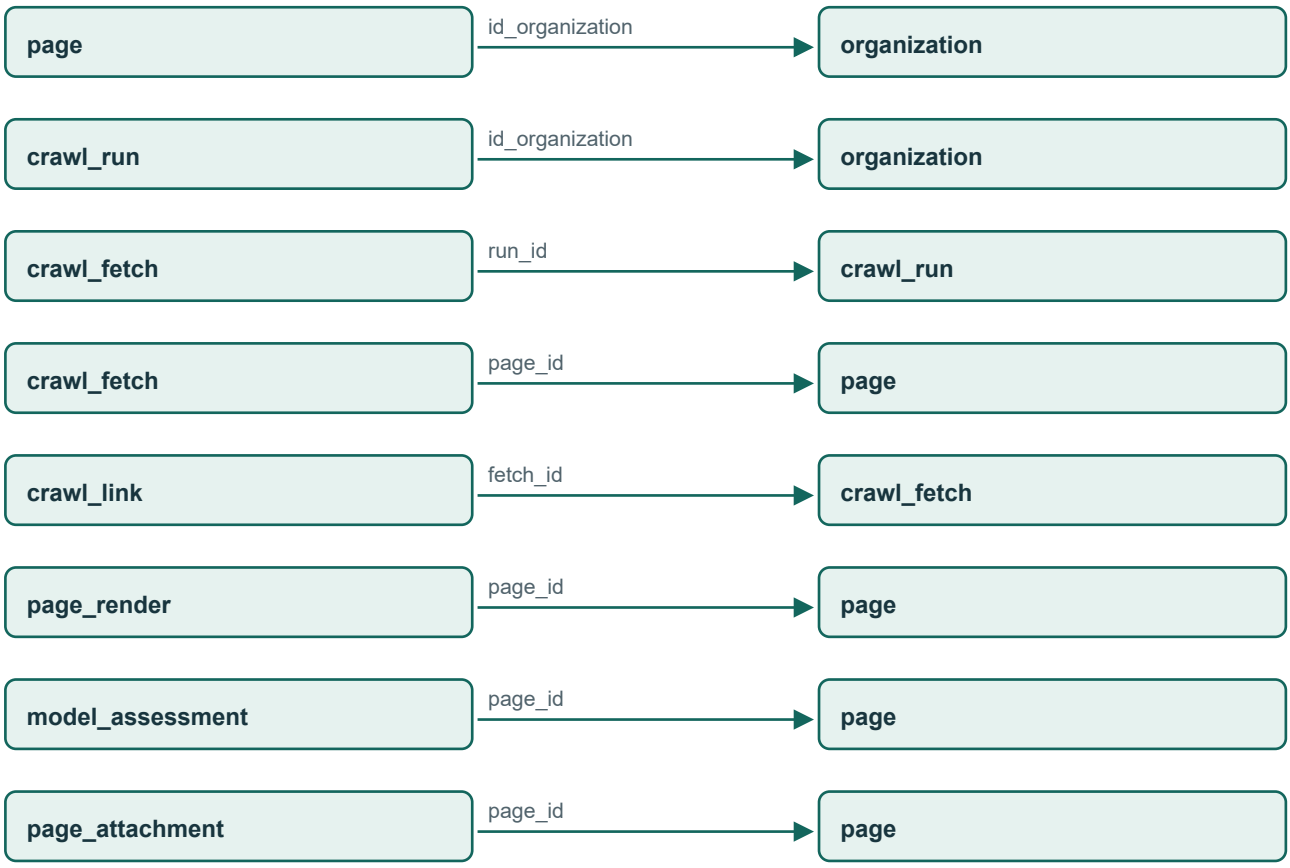
Сопровождение

Секрет владельца, приглашения и локальные резервные копии хранятся вне репозитория. Для повторного выпуска документации выполните python -m docs.build_manuals. Физическая схема извлекается из новой тестовой базы без чтения персональных настроек.

Устройство баз данных

Как читать стрелки

Ребро ведёт от дочерней записи к владельцу или объекту ссылки. Верхняя схема показывает основные связи архива. В нижней схеме логические ссылки экспресс-базы реализованы запросами приложения. Связь реестров определяется адресом сайта.



Экспресс-аудит



В таблицах portal_operation обеих баз независимо хранятся JSON-записи очереди. Время и текущий этап обновляются в контрольных точках. Полный перечень таблиц, полей и физических FK приведён далее.

Сводный архив • карта связей

Объявленные внешние ключи, связи 1–14 из 80. Слева дочерняя таблица, над стрелкой поле ссылки, справа таблица-владелец.



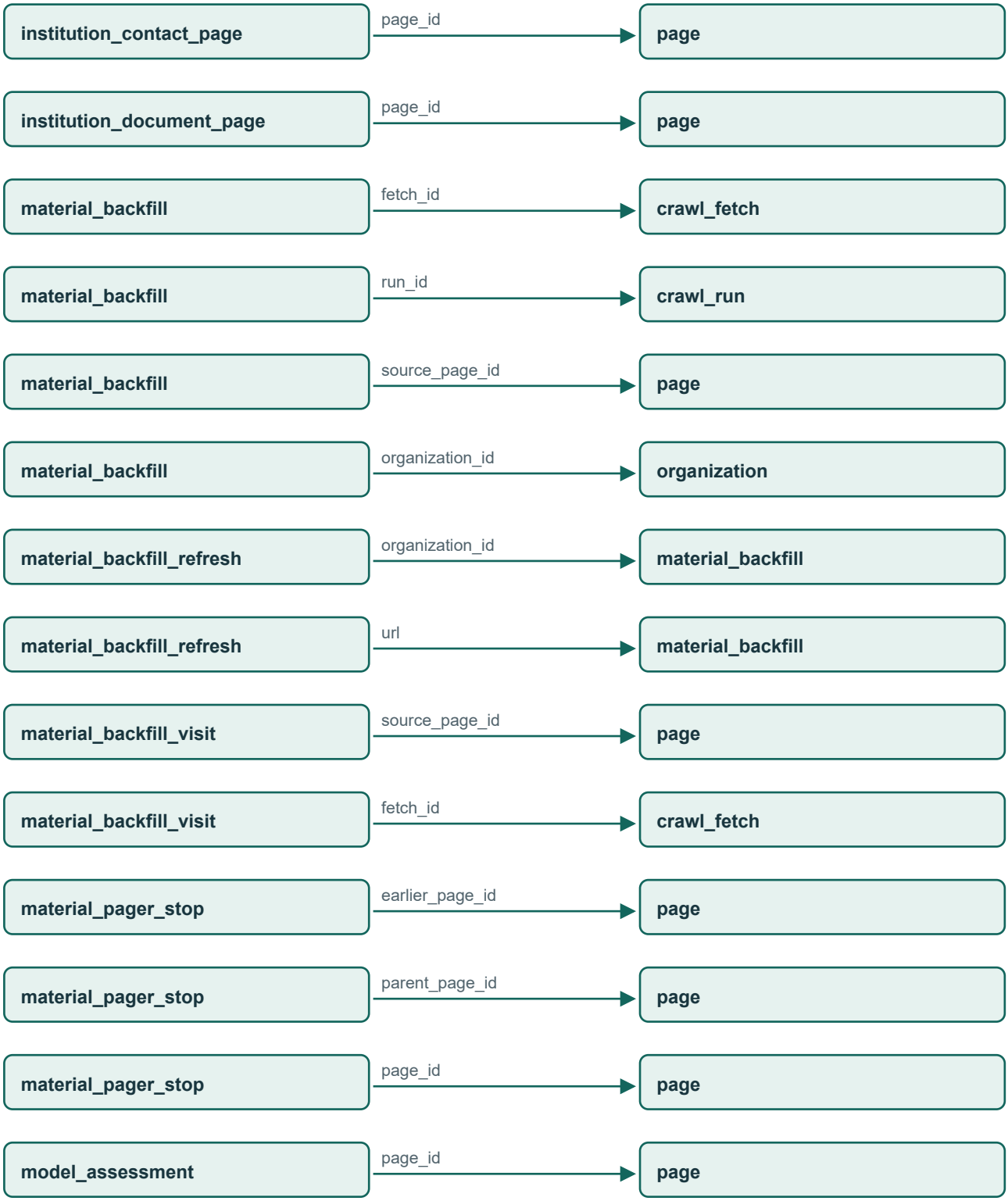
Сводный архив • карта связей

Объявленные внешние ключи, связи 15–28 из 80. Слева дочерняя таблица, над стрелкой поле ссылки, справа таблица-владелец.



Сводный архив • карта связей

Объявленные внешние ключи, связи 29–42 из 80. Слева дочерняя таблица, над стрелкой поле ссылки, справа таблица-владелец.



Сводный архив • карта связей

Объявленные внешние ключи, связи 43–56 из 80. Слева дочерняя таблица, над стрелкой поле ссылки, справа таблица-владелец.



Сводный архив • карта связей

Объявленные внешние ключи, связи 57–70 из 80. Слева дочерняя таблица, над стрелкой поле ссылки, справа таблица-владелец.



Сводный архив • карта связей

Объявленные внешние ключи, связи 71–80 из 80. Слева дочерняя таблица, над стрелкой поле ссылки, справа таблица-владелец.



Сводный архив • физическая схема

Как читать справочник

Таблиц: 69. PK обозначает позицию столбца в первичном ключе. ! обозначает обязательное значение NOT NULL. Отдельная строка перечисляет внешние ключи и их целевые поля. Источник описания — PRAGMA свежей базы, создаваемой кодом проекта.

app_setting

name TEXT PK1 · value TEXT !

attachment_download_lease

url TEXT PK1 · token TEXT ! · until INTEGER !

audit_worker_cycle

id INTEGER PK1 · started_at INTEGER ! · finished_at INTEGER · state TEXT ! · stage TEXT · settings TEXT ! · stages TEXT !

crawl_fetch

id INTEGER PK1 · run_id INTEGER ! · url TEXT ! · final_url TEXT · section TEXT ! · depth INTEGER ! · fetched_at INTEGER ! · http_status INTEGER · error TEXT · page_id INTEGER

FK page_id → page.id · run_id → crawl_run.id

crawl_heading

fetch_id INTEGER PK1 ! · section TEXT PK2 ! · label TEXT PK3 !

FK fetch_id → crawl_fetch.id

crawl_link

fetch_id INTEGER ! · url TEXT ! · label TEXT ! · section TEXT ! · relation TEXT ! · follow INTEGER !

FK fetch_id → crawl_fetch.id

crawl_run

id INTEGER PK1 · id_organization INTEGER ! · started_at INTEGER ! · finished_at INTEGER · state TEXT ! · purpose TEXT ! · max_depth INTEGER · max_pages INTEGER · stop_reason TEXT

FK id_organization → organization.id

factcheck_allowed_source

base_url TEXT PK1

factcheck_job

page_id INTEGER PK1 · run_id INTEGER ! · state TEXT ! · attempts INTEGER ! · lease_token TEXT · lease_until INTEGER · retry_after INTEGER · check_id INTEGER · outcome TEXT · error TEXT

FK check_id → factcheck_run.id · run_id → crawl_run.id · page_id → page.id

factcheck_job_history

id INTEGER PK1 · page_id INTEGER ! · allowlist_fingerprint TEXT · archived_at INTEGER ! · state TEXT ! · attempts INTEGER ! · check_id INTEGER · outcome TEXT · error TEXT · retry_after INTEGER · lease_until INTEGER

FK check_id → factcheck_run.id · page_id → factcheck_job.page_id

factcheck_job_policy

page_id INTEGER PK1 · allowlist_fingerprint TEXT !

FK page_id → factcheck_job.page_id

factcheck_run

id INTEGER PK1 · page_id INTEGER ! · checked_at INTEGER ! · query TEXT ! · minimum_sources INTEGER ! · source_count INTEGER ! · status TEXT ! · claim TEXT ! · searches TEXT !

FK page_id → page.id

factcheck_source

check_id INTEGER PK1 ! · ordinal INTEGER PK2 ! · url TEXT ! · final_url TEXT · publisher TEXT · status TEXT ! · evidence TEXT !

FK check_id → factcheck_run.id

factcheck_source_bootstrap

id INTEGER PK1

garden_building_photo

page_id INTEGER PK1 · status TEXT ! · image_url TEXT · evidence TEXT !

FK page_id → page.id

garden_home_contact

page_id INTEGER PK1 ! · field TEXT PK2 ! · status TEXT ! · evidence TEXT !

FK page_id → page.id

garden_home_section

page_id INTEGER PK1 ! · section TEXT PK2 ! · heading_found INTEGER ! · block_status TEXT ! · navigation_count INTEGER ! · listing_count INTEGER ! · card_count INTEGER ! · evidence TEXT !

FK page_id → page.id

home_analysis_problem

page_id INTEGER PK1 · attempted_at INTEGER ! · error TEXT !

FK page_id → page.id

home_canteen_photo

page_id INTEGER PK1 · status TEXT ! · image_url TEXT · alt TEXT · evidence TEXT !

FK page_id → page.id

home_event_block

page_id INTEGER PK1 · page_kind TEXT ! · block_status TEXT ! · evidence TEXT !

FK page_id → page.id

home_event_link

page_id INTEGER PK1 ! · ordinal INTEGER PK2 ! · kind TEXT ! · url TEXT · title TEXT · date_text TEXT · date_start TEXT · date_end TEXT · image_url TEXT · evidence TEXT !

FK page_id → home_event_block.page_id

home_image_asset

sha256 TEXT PK1 · mime TEXT ! · width INTEGER ! · height INTEGER ! · data BLOB !

home_image_download

page_id INTEGER PK1 ! · image_url TEXT PK2 ! · fetched_at INTEGER ! · final_url TEXT ! · http_status INTEGER · status TEXT ! · error TEXT · sha256 TEXT

FK sha256 → home_image_asset.sha256 · page_id → page.id

home_main_photo

page_id INTEGER PK1 · status TEXT ! · image_url TEXT · alt TEXT · evidence TEXT !

FK page_id → page.id

home_school_description

page_id INTEGER PK1 · status TEXT ! · text TEXT · char_count INTEGER · evidence TEXT !

FK page_id → page.id

home_term

category TEXT PK1 ! · term TEXT PK2 !

FK category → home_term_category.category

home_term_category

category TEXT PK1

inactive_organization

organization_id INTEGER PK1

FK organization_id → organization.id

institution_audit

run_id INTEGER PK1 · home_page_id INTEGER · evidence TEXT !

FK home_page_id → page.id · run_id → crawl_run.id

institution_contact_page

page_id INTEGER PK1 · section TEXT ! · source TEXT ! · observations TEXT ! · section_links TEXT !

FK page_id → page.id

institution_document_page

page_id INTEGER PK1 · section TEXT ! · catalog INTEGER ! · source TEXT ! · evidence TEXT !

FK page_id → page.id

material_backfill

organization_id INTEGER PK1 ! · url TEXT PK2 ! · source_page_id INTEGER ! · section TEXT ! · relation TEXT ! · depth INTEGER ! · priority INTEGER ! · state TEXT ! · run_id INTEGER · fetch_id INTEGER · lease_until INTEGER · error TEXT

FK fetch_id → crawl_fetch.id · run_id → crawl_run.id · source_page_id → page.id · organization_id → organization.id

material_backfill_refresh

organization_id INTEGER PK1 ! · url TEXT PK2 ! · cutoff INTEGER !

FK organization_id → material_backfill.organization_id · url → material_backfill.url

material_backfill_visit

fetch_id INTEGER PK1 · source_page_id INTEGER ! · relation TEXT !

FK source_page_id → page.id · fetch_id → crawl_fetch.id

material_pager_stop

page_id INTEGER PK1 · parent_page_id INTEGER ! · earlier_page_id INTEGER ! · reason TEXT !

FK earlier_page_id → page.id · parent_page_id → page.id · page_id → page.id

model_assessment

id INTEGER PK1 · page_id INTEGER ! · check_id TEXT ! · created_at INTEGER ! · provider TEXT ! · endpoint TEXT ! · model TEXT ! · outcome TEXT ! · request TEXT ! · response TEXT · error TEXT · elapsed_seconds REAL ! · usage_json TEXT

FK page_id → page.id

model_batch_result

page_id INTEGER PK1 ! · check_id TEXT PK2 ! · profile_key TEXT PK3 ! · input_key TEXT PK4 ! · assessment_id INTEGER !

FK assessment_id → model_assessment.id · page_id → page.id

model_evidence_pass

assessment_id INTEGER PK1 ! · ordinal INTEGER PK2 ! · material_ids TEXT ! · outcome TEXT · evidence TEXT · error TEXT · usage_json TEXT · raw_response TEXT

FK assessment_id → model_assessment.id

model_profile

provider TEXT PK1 · settings TEXT !

model_review_guard

assessment_id INTEGER PK1 · original_response TEXT ! · reason TEXT !

FK assessment_id → model_assessment.id

model_visual_verification

assessment_id INTEGER PK1 · status TEXT ! · guided_pass TEXT ! · target_pass TEXT

FK assessment_id → model_assessment.id

news_page

page_id INTEGER PK1 · page_kind TEXT ! · source TEXT ! · order_status TEXT ! · card_count INTEGER ! · evidence TEXT !

FK page_id → page.id

news_preview

page_id INTEGER PK1 ! · ordinal INTEGER PK2 ! · url TEXT ! · title TEXT ! · date_status TEXT ! · date_text TEXT · date_iso TEXT · evidence TEXT !

FK page_id → news_page.page_id

news_window_date

organization_id INTEGER PK1 ! · url TEXT PK2 ! · page_id INTEGER PK3 ! · ordinal INTEGER PK4 ! · date_iso TEXT · reason TEXT !

FK page_id → page.id

news_window_page

page_id INTEGER PK1 ! · organization_id INTEGER ! · months INTEGER PK2 ! · day TEXT PK3 ! · start_day TEXT ! · section_url TEXT ! · series TEXT · component TEXT · mode TEXT · position INTEGER · newest TEXT · boundary_url TEXT · boundary_date TEXT · reason TEXT !

FK page_id → page.id

news_window_skip

organization_id INTEGER PK1 ! · url TEXT PK2 ! · months INTEGER PK3 ! · day TEXT PK4 ! · start_day TEXT ! · page_id INTEGER ! · reason TEXT ! · boundary_url TEXT · date_iso TEXT

FK page_id → page.id

organization

id INTEGER PK1 · name TEXT ! · site_url TEXT ! · kind TEXT !

page

id INTEGER PK1 · url TEXT ! · id_organization INTEGER ! · date_loaded INTEGER ! · text_page TEXT !

FK id_organization → organization.id

page_attachment

id INTEGER PK1 · page_id INTEGER ! · url TEXT ! · title TEXT ! · role TEXT ! · format_hint TEXT ! · state TEXT ! · fetched_at INTEGER · final_url TEXT · http_status INTEGER · format TEXT · sha256 TEXT · byte_size INTEGER · error TEXT · attempts INTEGER !

FK page_id → page.id

page_index_issue

page_id INTEGER PK1 ! · analyzer TEXT PK2 ! · error TEXT !

FK page_id → page.id

page_passport

page_id INTEGER PK1 · page_kind TEXT ! · title TEXT ! · headings TEXT ! · summary TEXT ! · checks TEXT !

FK page_id → page.id

page_render

page_id INTEGER PK1 · captured_at INTEGER ! · url TEXT ! · status TEXT ! · viewport_width INTEGER ! · viewport_height INTEGER ! · css_loaded INTEGER · details TEXT ! · rendered_html TEXT · screenshot_mime TEXT · screenshot BLOB · full_screenshot BLOB · pdf BLOB

FK page_id → page.id

page_revisit

original_page_id INTEGER ! · fetch_id INTEGER PK1 · relation TEXT !

FK fetch_id → crawl_fetch.id · original_page_id → page.id

portal_operation

id INTEGER PK1 · record TEXT !

retention_disk_queue

sha256 TEXT PK1 ! · format TEXT PK2 ! · byte_size INTEGER !

retention_item

page_id INTEGER PK1 ! · kind TEXT PK2 ! · source TEXT PK3 ! · removed_at INTEGER ! · replacement_page_id INTEGER ! · byte_size INTEGER ! · sha256 TEXT

FK replacement_page_id → page.id · page_id → page.id

retention_policy

id INTEGER PK1 · enabled INTEGER ! · days INTEGER !

school_event_quarter

run_id INTEGER PK1 ! · period TEXT PK2 ! · start_date TEXT ! · end_date TEXT ! · status TEXT ! · dated_events INTEGER ! · evidence TEXT !

FK run_id → crawl_run.id

school_schedule_page

page_id INTEGER PK1 · source TEXT ! · status TEXT ! · evidence TEXT !

FK page_id → page.id

site_navigator_page

page_id INTEGER PK1 · is_navigator_page INTEGER ! · evidence TEXT !

FK page_id → page.id

staff_cross_org_evidence

observation_id INTEGER PK1 · name_key TEXT ! · organization_id INTEGER !

FK name_key → staff_cross_org_match.name_key · organization_id → staff_cross_org_match.organization_id · observation_id → staff_observation.id

staff_cross_org_match

name_key TEXT PK1 ! · organization_id INTEGER PK2 ! · run_id INTEGER ! · representative_observation_id INTEGER · organization_count INTEGER ! · observation_count INTEGER ! · status TEXT !

FK representative_observation_id → staff_observation.id · run_id → crawl_run.id · organization_id → organization.id

staff_element

id INTEGER PK1 · observation_id INTEGER ! · field TEXT ! · selector TEXT ! · expected_value TEXT ! · visual_status TEXT ! · visual_details TEXT · checked_at INTEGER · render_source TEXT · render_url TEXT · css_loaded INTEGER · viewport_width INTEGER · viewport_height INTEGER

FK observation_id → staff_observation.id

staff_observation

id INTEGER PK1 · page_id INTEGER ! · card_selector TEXT ! · full_name TEXT · role TEXT · is_head INTEGER ! · photo_url TEXT · photo_state TEXT ! · education TEXT · email TEXT · phone TEXT · profile_url TEXT · method TEXT ! · duplicate_group TEXT · duplicate_match TEXT ! · duplicate_card_count INTEGER · duplicate_page_count INTEGER · duplicate_cards_on_page INTEGER · duplicate_roles_differ INTEGER · duplicate_needs_review INTEGER

FK page_id → page.id

staff_scan_run

run_id INTEGER PK1 · scanned_at INTEGER ! · pages INTEGER ! · observations INTEGER !

FK run_id → crawl_run.id

timeline_event_news

run_id INTEGER PK1 ! · event_key TEXT PK2 ! · event_title TEXT · date_start TEXT · date_end TEXT · status TEXT ! · evidence TEXT !

FK run_id → crawl_run.id

timeline_gap

run_id INTEGER PK1 ! · page_id INTEGER PK2 ! · ordinal INTEGER PK3 ! · next_page_id INTEGER PK4 ! · next_ordinal INTEGER PK5 ! · days INTEGER · status TEXT ! · evidence TEXT !

FK next_page_id → page.id · page_id → page.id · run_id → crawl_run.id

timeline_publication

run_id INTEGER PK1 ! · url TEXT PK2 ! · page_id INTEGER · title TEXT ! · published_start TEXT · published_end TEXT · event_start TEXT · event_end TEXT · status TEXT ! · evidence TEXT !

FK page_id → page.id · run_id → crawl_run.id

timeline_run

run_id INTEGER PK1 · summary TEXT !

FK run_id → crawl_run.id

Представления

active_organization, available_home_image_download, available_page_attachment, home_current_image_use, home_image_reference, home_reused_asset, home_reused_image, institution_contact_observation, institution_document_link, news_model_page, school_event_date_issue, staff_cross_org_source

Экспресс-аудит • физическая схема

Как читать справочник

Таблиц: 8. PK обозначает позицию столбца в первичном ключе. ! обозначает обязательное значение NOT NULL. Отдельная строка перечисляет внешние ключи и их целевые поля. Источник описания — PRAGMA свежей базы, создаваемой кодом проекта.

audit_rules

id INTEGER PK1 · rule_type TEXT ! · rule_key TEXT ! · rule_value TEXT ! · active INTEGER !

findings

id INTEGER PK1 · run_id INTEGER ! · site_id INTEGER ! · agent TEXT · severity TEXT · code TEXT · message TEXT · url TEXT · recommendation TEXT

photo_targets

id INTEGER PK1 · section_key TEXT ! · section_label TEXT ! · page_keywords TEXT ! · require_faces INTEGER ! · active INTEGER !

portal_operation

id INTEGER PK1 · record TEXT !

run_sites

id INTEGER PK1 · run_id INTEGER ! · site_id INTEGER ! · site_name TEXT · site_url TEXT · score INTEGER · critical INTEGER · high INTEGER · medium INTEGER · low INTEGER · latest_news TEXT · future_events INTEGER · metrics_json TEXT · llm_summary TEXT · ai_explanation TEXT · checked_at TEXT

runs

id INTEGER PK1 · started_at TEXT ! · finished_at TEXT · status TEXT ! · sites_total INTEGER ! · avg_score REAL

settings

key TEXT PK1 · value TEXT

sites

id INTEGER PK1 · name TEXT ! · url TEXT ! · type TEXT ! · active INTEGER ! · email TEXT